
● SAVOL 51:

[NUMPY] 1 DAN 100 GACHA SONLARDAN IBORAT 10×10 MATRITSA YARATING. HAR QATOR VA HAR USTUNNING O'RTACHA QIYMATINI, UMUMIY MAKSIMUM VA MINIMUMNI HISOBLANG VA EKRANGA CHIQARING.

```
import numpy as np

# =====
# MATRITSA YARATISH
# =====

# np.arange(1, 101) → [1, 2, 3, ..., 100] – 100 ta son
# .reshape(10, 10) → 1D massivni 10 qator × 10 ustun ga aylantirish
matritsa = np.arange(1, 101).reshape(10, 10)

print("=" * 55)
print(" 10×10 MATRITSA:")
print("=" * 55)

# np.set_printoptions → chiqish formatini sozlash
# linewidth → bir qatorda nechta belgi sig'adi
np.set_printoptions(linewidth=60)
print(matritsa)

# =====
# HAR QATORNING O'RTACHASI
# =====

# axis=1 → har QATOR bo'ylab hisoblash (gorizontal)
# axis=0 → har USTUN bo'ylab hisoblash (vertikal)
# Natija: 10 ta qiymat (har qator uchun bitta)
qator_ortacha = matritsa.mean(axis=1)

print("\n" + "=" * 55)
print("  HAR QATORNING O'RTACHA QIYMATI (axis=1):")
print("=" * 55)
for i, qiymat in enumerate(qator_ortacha):
    print(f" {i+1:2}-qator: {qiymat:.1f}")

# =====
# HAR USTUNNING O'RTACHASI
# =====

# axis=0 → har USTUN bo'ylab hisoblash (vertikal)
# Natija: 10 ta qiymat (har ustun uchun bitta)
ustun_ortacha = matritsa.mean(axis=0)

print("\n" + "=" * 55)
print("  HAR USTUNNING O'RTACHA QIYMATI (axis=0):")
print("=" * 55)
for j, qiymat in enumerate(ustun_ortacha):
    print(f" {j+1:2}-ustun: {qiymat:.1f}")
```

```

# =====
# UMUMIY MAKSIMUM VA MINIMUM
# =====

# .max() / .min() – butun matritsa bo'ylab eng katta/kichik son
# np.argmax() → maksimum elementning tekis (flat) indeksi
# np.unravel_index() → tekis indeksni (qator, ustun) ga aylantirish
umumiy_max = matritsa.max()
umumiy_min = matritsa.min()

max_index = np.unravel_index(matritsa.argmax(), matritsa.shape)
min_index = np.unravel_index(matritsa.argmin(), matritsa.shape)

print("\n" + "=" * 55)
print("  UMUMIY MAKSIMUM VA MINIMUM:")
print("=" * 55)
print(f"  Maksimum: {umumiy_max}"
      f" → [{max_index[0]+1}-qator, {max_index[1]+1}-ustun]")
print(f"  Minimum: {umumiy_min}"
      f" → [{min_index[0]+1}-qator, {min_index[1]+1}-ustun]")
print("=" * 55)

```

Chiqish Natijasi

```

=====
10×10 MATRITSA:
=====
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]
 [21 22 23 24 25 26 27 28 29 30]
 [31 32 33 34 35 36 37 38 39 40]
 [41 42 43 44 45 46 47 48 49 50]
 [51 52 53 54 55 56 57 58 59 60]
 [61 62 63 64 65 66 67 68 69 70]
 [71 72 73 74 75 76 77 78 79 80]
 [81 82 83 84 85 86 87 88 89 90]
 [91 92 93 94 95 96 97 98 99 100]]

=====
HAR QATORNING O'RTACHA QIYMATI (axis=1):
=====
1-qator:  5.5
2-qator: 15.5
3-qator: 25.5
4-qator: 35.5
5-qator: 45.5
6-qator: 55.5
7-qator: 65.5
8-qator: 75.5
9-qator: 85.5
10-qator: 95.5

=====
HAR USTUNNING O'RTACHA QIYMATI (axis=0):

```

```
=====
1-ustun: 46.0
2-ustun: 47.0
3-ustun: 48.0
4-ustun: 49.0
5-ustun: 50.0
6-ustun: 51.0
7-ustun: 52.0
8-ustun: 53.0
9-ustun: 54.0
10-ustun: 55.0

=====
UMUMIY MAKSIMUM VA MINIMUM:
=====
Maksimum: 100 → [10-qator, 10-ustun]
Minimum:   1 → [ 1-qator,  1-ustun]
=====
```

Dastur Tuzilishi

```
Dastur
├─ Matritsa yaratish
│   ├── np.arange(1, 101) → 1D: [1..100]
│   └── .reshape(10, 10) → 2D: 10×10
├─ Qator o'rtachasi
│   └── .mean(axis=1) → 10 ta qiymat
├─ Ustun o'rtachasi
│   └── .mean(axis=0) → 10 ta qiymat
└─ Umumiy max / min
    ├── .max() / .min() → bitta qiymat
    └── np.unravel_index() → (qator, ustun) joylashuvi
```

Asosiy Texnik Tushunchalar

```
reshape(10, 10):
[1, 2, ..., 100] → 10 qator × 10 ustun
Shart: elementlar soni mos kelishi kerak
100 = 10 × 10 ✓
```

```
axis parametri:
axis=0 → USTUN bo'ylab (pastga qarab)
        har ustundan 1 natija → shape: (10,)
axis=1 → QATOR bo'ylab (o'ngga qarab)
        har qatordan 1 natija → shape: (10,)
```

Vizual:

```
axis=0 ↓↓↓          axis=1 →→→
[ 1  2  3]          [ 1  2  3] → o'rtacha: 2.0
[ 4  5  6]          [ 4  5  6] → o'rtacha: 5.0
↓  ↓  ↓
2.5 3.5 4.5
```

```
unravel_index():  
    matritsa.argmax() → tekis indeks: 99 (flat)  
    np.unravel_index(99, (10,10)) → (9, 9) (qator, ustun)  
    Chunki: tekis indeks 99 → 10-qator, 10-ustun
```

💡 **Eslab qol:** `axis` parametrini **teskari** o'ylash oson: `axis=1` deb yozsang, o'sha o'q **yo'q qilinadi** — ya'ni 10 ustun bitta qiymatga **siqiladi**, qatorlar saqlanadi. `axis=0` da esa 10 qator siqilib, ustunlar saqlanadi. "Qaysi o'q bo'ylab hisoblayman?" emas, "**qaysi o'qni yo'q qilaman?**" deb o'yla.

● SAVOL 52:

[PANDAS] 5 TA TALABA VA 4 TA FAN BAHOSIDAN IBORAT DATAFRAME TUZING. HAR BIR TALABANING O'RTACHA BALINI HISOBLANG VA "A'LO/YAXSHI/QONIQARLI" USTUNINI QO'SHING.

```
import pandas as pd  
  
# =====  
# DATAFRAME YARATISH  
# =====  
  
# Dictionary dan DataFrame yaratish:  
# Kalit → ustun nomi  
# Qiymat → o'sha ustundagi qiymatlar ro'yxati  
malumot = {  
    "Talaba": ["Ali", "Vali", "Hasan", "Zulfiya", "Malika"],  
    "Matematika": [90, 75, 60, 95, 82],  
    "Fizika": [85, 70, 55, 88, 78],  
    "Ingliz tili": [92, 80, 65, 97, 74],  
    "Informatika": [88, 65, 58, 91, 86],  
}  
  
df = pd.DataFrame(malumot)  
  
# "Talaba" ustunini indeks qilib belgilash  
# Shunda qator raqami (0,1,2...) o'rniga ism ko'rinadi  
df = df.set_index("Talaba")  
  
print("=" * 58)  
print(" DASTLABKI DATAFRAME:")  
print("=" * 58)  
print(df)  
  
# =====  
# O'RTACHA BAL HISOBLASH  
# =====  
  
# axis=1 → har QATOR bo'ylab o'rtacha (har talaba uchun)  
# round(2) → 2 kasr raqamgacha yaxlitlash  
df["O'rtacha"] = df.mean(axis=1).round(2)
```

```

print("\n" + "=" * 58)
print(" O'RTACHA BAL QO'SHILDI:")
print("=" * 58)
print(df)

```

```

# =====
# BAHO USTUNINI QO'SHISH – apply() + lambda
# =====

```

```

# Baho mezoni:
# 85 va yuqori → "A'lo"
# 70 – 84 → "Yaxshi"
# 70 dan past → "Qoniqarli"

```

```

def baho_belgilash(ortacha):
    """
    O'rtacha balga qarab baho qaytaradi.
    Bu funksiya har bir qator uchun chaqiriladi.
    """
    if ortacha >= 85:
        return "A'lo"
    elif ortacha >= 70:
        return "Yaxshi"
    else:
        return "Qoniqarli"

```

```

# apply() → har bir qiymatga funksiya qo'llash
# "O'rtacha" ustunidagi har bir qiymat baho_belgilash() ga beriladi
df["Baho"] = df["O'rtacha"].apply(baho_belgilash)

```

```

print("\n" + "=" * 58)
print(" BAHU USTUNI QO'SHILDI:")
print("=" * 58)
print(df)

```

```

# =====
# QO'SHIMCHA TAHLIL
# =====

```

```

print("\n" + "=" * 58)
print(" BAHU BO'YICHA STATISTIKA:")
print("=" * 58)

```

```

# value_counts() → har bir bahoning necha marta uchrashini sanash
baho_soni = df["Baho"].value_counts()
for baho, son in baho_soni.items():
    print(f" {baho:<12}: {son} ta talaba")

```

```

print("\n" + "=" * 58)
print(" ENG YUQORI VA PAST O'RTACHA:")
print("=" * 58)

```

```

# idxmax() / idxmin() → eng katta/kichik qiymatning indeksini qaytaradi
# Indeks = talaba ismi (set_index dan keyin)
eng_yaxshi = df["O'rtacha"].idxmax()
eng_past = df["O'rtacha"].idxmin()

```

```

print(f" Eng yuqori: {eng_yaxshi:<10}"
      f"> {df.loc[eng_yaxshi, '0\'rtacha']}]")
print(f" Eng past: {eng_past:<10}"
      f"> {df.loc[eng_past, '0\'rtacha']}]")

print("\n" + "=" * 58)
print(" HAR FAN BO'YICHA SINF O'RTACHASI:")
print("=" * 58)

# Faqat raqamli ustunlar (baho va o'rtacha ustunlarsiz)
fanlar = ["Matematika", "Fizika", "Ingliz tili", "Informatika"]
for fan in fanlar:
    print(f" {fan:<14}: {df[fan].mean():.1f}")

print("=" * 58)

```

Chiqish Natijasi

```

=====
DASTLABKI DATAFRAME:
=====

```

	Matematika	Fizika	Ingliz tili	Informatika
Talaba				
Ali	90	85	92	88
Vali	75	70	80	65
Hasan	60	55	65	58
Zulfiya	95	88	97	91
Malika	82	78	74	86

```

=====
O'RTACHA BAL QO'SHILDI:
=====

```

	Matematika	Fizika	Ingliz tili	Informatika	O'rtacha
Talaba					
Ali	90	85	92	88	88.75
Vali	75	70	80	65	72.50
Hasan	60	55	65	58	59.50
Zulfiya	95	88	97	91	92.75
Malika	82	78	74	86	80.00

```

=====
BAHO USTUNI QO'SHILDI:
=====

```

	Matematika	Fizika	Ingliz tili	Informatika	O'rtacha	Baho
Talaba						
Ali	90	85	92	88	88.75	A'lo
Vali	75	70	80	65	72.50	Yaxshi
Hasan	60	55	65	58	59.50	Qoniqarli
Zulfiya	95	88	97	91	92.75	A'lo
Malika	82	78	74	86	80.00	Yaxshi

```

=====
BAHO BO'YICHA STATISTIKA:
=====

```

A'lo : 2 ta talaba

Yaxshi : 2 ta talaba
Qoniqarli : 1 ta talaba

=====

ENG YUQORI VA PAST O'RTACHA:

=====

Eng yuqori: Zulfiya → 92.75
Eng past: Hasan → 59.50

=====

HAR FAN BO'YICHA SINIF O'RTACHASI:

=====

Matematika : 80.4
Fizika : 75.2
Ingliz tili : 81.6
Informatika : 77.6

=====

Dastur Tuzilishi

Dastur

- └ DataFrame yaratish
 - └ pd.DataFrame(dict) → jadval
 - └ set_index("Talaba") → ism → indeks
- └ O'rtacha hisoblash
 - └ .mean(axis=1) → har qator bo'ylab
- └ Baho ustuni
 - └ baho_belgilash() → mezon funksiyasi
 - └ .apply(funksiya) → har qiymatga qo'llash
- └ Tahlil
 - └ .value_counts() → baho bo'yicha sanoq
 - └ .idxmax() / .idxmin() → eng yaxshi/past talaba
 - └ .mean() → fan bo'yicha o'rtacha

Asosiy Texnik Tushunchalar

set_index("Talaba"):

Talaba ustunini qator indeksiga aylantirish
Afzallik: df.loc["Ali"] → to'g'ridan qatorni olish

mean(axis=1):

axis=0 → ustun bo'ylab (barcha talabalar → 1 qiymat)
axis=1 → qator bo'ylab (barcha fanlar → 1 qiymat) ✓

apply():

df["O'rtacha"].apply(baho_belgilash)

Har bir qiymat

Shu funksiyaga beriladi

[88.75, 72.5, 59.5, 92.75, 80.0]

↓ ↓ ↓ ↓ ↓

[A'lo, Yaxshi, Qon., A'lo, Yaxshi]

```

Lambda bilan ham yozsa bo'ladi (qisqaroq):
df["Baho"] = df["O'rtacha"].apply(
    lambda x: "A'lo" if x >= 85
              else "Yaxshi" if x >= 70
              else "Qoniqarli"
)

```

```

loc[] – indeks bo'yicha qiymat olish:
df.loc["Ali", "O'rtacha"] → 88.75
df.loc["Zulfiya"]         → Zulfiyaning butun qatori

```

💡 **Eslab qol:** `apply()` har bir element uchun Python funksiyasini chaqiradi — qulay, lekin katta DataFramelarda **sekin**. Tezroq yechim: `pd.cut()` yoki `np.select()` — ular vektorlashtirilgan va ming-million qatorda ham tez ishlaydi. Kichik ma'lumotlarda `apply()` tushunarli va to'g'ri tanlov.

● SAVOL 53:

[MATPLOTLIB] UCHTA SHAHAR UCHUN 12 OYLIK HARORAT MA'LUMOTLARINI BIR GRAFIKDA KO'RSATING. HAR SHAHAR UCHUN TURLI RANG VA MARKER ISHLATIB, LEGEND, SARLAVHA VA O'Q NOMLARI QO'SHING.

```

import matplotlib.pyplot as plt
import numpy as np

# =====
# MA'LUMOTLAR
# =====

oylar = [
    "Yan", "Fev", "Mar", "Apr",
    "May", "Iyn", "Iyl", "Avg",
    "Sen", "Okt", "Noy", "Dek"
]

# Har shahar uchun 12 oylik harorat (°C)
haroratlar = {
    "Toshkent": [-1, 2, 9, 17, 22, 27, 29, 28, 22, 14, 7, 1],
    "Moscow":   [-9, -8, -2, 6, 13, 17, 19, 17, 11, 4, -3, -7],
    "Dubai":    [19, 20, 23, 27, 32, 34, 36, 37, 33, 29, 25, 21],
}

# X o'qi uchun raqamli indekslar (0..11)
# plt.xticks() da oylar nomi bosiladi
x = np.arange(len(oylar))

# =====
# GRAFIK SOZLAMALARI
# =====

# figsize → grafik o'lchami dyumda (kenglik, balandlik)
# dpi      → piksel zichligi (yuqori = aniqroq)

```

```

fig, ax = plt.subplots(figsize=(12, 6), dpi=100)

# Fon rangi
fig.patch.set_facecolor("#0f1117") # Oyna foni
ax.set_facecolor("#1a1d27")        # Grafik maydoni foni

# =====
# HAR SHAHAR UCHUN CHIZIQ CHIZISH
# =====

# Har shahar uchun alohida rang, marker va chiziq uslubi
shahar_sozlash = {
    "Toshkent": {
        "color":      "#7c85f3",    # Binafsha
        "marker":      "o",          # Doira marker
        "linestyle":    "-",          # To'liq chiziq
        "linewidth":    2.5,
        "markersize": 7,
    },
    "Moscow": {
        "color":      "#f38ba8",    # Qizilimtir
        "marker":      "s",          # Kvadrat marker
        "linestyle":    "--",        # Kasma chiziq
        "linewidth":    2.5,
        "markersize": 7,
    },
    "Dubai": {
        "color":      "#a6e3a1",    # Yashil
        "marker":      "^",          # Uchburchak marker
        "linestyle":    "-.",        # Nuqta-chiziq
        "linewidth":    2.5,
        "markersize": 8,
    },
}

for shahar, harorat in haroratlar.items():
    sozlash = shahar_sozlash[shahar]

    # plot() → chiziq + marker chizish
    # zorder → ustma-ust chizilganda qaysi ustda turishi
    ax.plot(
        x, harorat,
        label      = shahar,
        color       = sozlash["color"],
        marker      = sozlash["marker"],
        linestyle   = sozlash["linestyle"],
        linewidth   = sozlash["linewidth"],
        markersize  = sozlash["markersize"],
        zorder      = 3,
    )

    # Har marker ustiga harorat qiymati yozish
    for i, t in enumerate(harorat):
        ax.annotate(
            f"{t}°",
            xy      = (i, t),
            xytext   = (0, 10),          # Markerdan 10px yuqori

```

```

        textcoords = "offset points",
        ha         = "center",
        fontsize   = 7.5,
        color      = sozlash["color"],
        alpha      = 0.85,
    )

# =====
# NOLDAN O'TUVCHI GORIZONTAL CHIZIQ
# =====

# axhline → gorizontal to'g'ri chiziq (axis horizontal line)
# y=0      → 0°C darajasida
ax.axhline(
    y         = 0,
    color     = "#45475a",
    linestyle = ":",
    linewidth = 1.2,
    label     = "0°C (muzlash)",
    zorder    = 2,
)

# =====
# O'Q SOZLAMALARI
# =====

# X o'qi: raqam o'rniga oy nomlari
ax.set_xticks(x)
ax.set_xticklabels(oylar, fontsize=11, color="#cdd6f4")

# Y o'qi: harorat oralig'i va qadami
ax.set_yticks(range(-15, 45, 5))
ax.yaxis.set_tick_params(labelsize=10, labelcolor="#cdd6f4")

# O'q chegaralari – ma'lumot + yozuvlar uchun bo'sh joy
ax.set_xlim(-0.5, 11.5)
ax.set_ylim(-18, 45)

# O'q nomlari
ax.set_xlabel(
    "Oylar",
    fontsize = 13,
    color    = "#8892b0",
    labelpad = 12,
)
ax.set_ylabel(
    "Harorat (°C)",
    fontsize = 13,
    color    = "#8892b0",
    labelpad = 12,
)

# =====
# SARLAVHA
# =====

ax.set_title(

```

```

        "Uchta Shahar – 12 Oylik Harorat (2024)",
        fontsize = 16,
        fontweight = "bold",
        color = "#e2e8f0",
        pad = 18,
    )

# =====
# LEGEND (IZOH)
# =====

# loc="upper left" → chapdan yuqori burchakda
# framealpha → legend foni shaffofligi
# edgecolor → legend chegarasi rangi
legend = ax.legend(
    loc = "upper left",
    fontsize = 11,
    framealpha = 0.85,
    edgecolor = "#363a52",
    facecolor = "#252836",
    labelcolor = "#cdd6f4",
)

# =====
# GRID (TO'R CHIZIQLAR)
# =====

# axis="y" → faqat gorizonttal to'r (o'qish oson)
# alpha → shaffoflik (0=ko'rinmas, 1=to'liq)
ax.grid(
    axis = "y",
    color = "#2d3148",
    linestyle = "--",
    linewidth = 0.8,
    alpha = 0.7,
    zorder = 1,
)

# O'q ramkasi (spine) rangini o'zgartirish
for spine in ax.spines.values():
    spine.set_edgecolor("#2d3148")

# =====
# GRAFIK SAQLASH VA KO'RSATISH
# =====

# tight_layout → elementlar bir-biriga tegib qolmasin
plt.tight_layout()

# Faylga saqlash (ixtiyoriy)
# plt.savefig("harorat.png", dpi=150, bbox_inches="tight")

plt.show()

```

Dastur Tuzilishi

Dastur		
└─ Ma'lumotlar		
└─ oylar[]	→ 12 oy nomi	
└─ haroratlar{}	→ 3 shahar × 12 qiymat	
└─ Grafik sozlash		
└─ plt.subplots()	→ fig, ax yaratish	
└─ facecolor	→ fon rangi	
└─ Chiziqlar		
└─ ax.plot()	→ rang, marker, linestyle	
└─ ax.annotate()	→ har nuqtaga qiymat yozish	
└─ O'q sozlash		
└─ set_xticks()	→ oy nomlari	
└─ set_xlabel/ylabel()	→ o'q sarlavhalari	
└─ set_xlim/ylim()	→ ko'rinish oralig'i	
└─ Qo'shimchalar		
└─ ax.legend()	→ izoh qutisi	
└─ ax.grid()	→ to'r chiziqlar	
└─ ax.axhline(y=0)	→ 0°C belgisi	
└─ plt.show()	→ ekranga chiqarish	

Asosiy Texnik Tushunchalar

plt.subplots() vs plt.plot():

plt.plot() → tezkor, sozlash cheklangan
 plt.subplots() → fig + ax → to'liq nazorat ✓
 fig → oyna (o'lcham, saqlash)
 ax → grafik maydoni (chiziqlar, o'qlar, legend)

Markerlar:

"o" → ● doira
 "s" → ■ kvadrat
 "^" → ▲ uchburchak (yuqoriga)
 "v" → ▼ uchburchak (pastga)
 "D" → ◆ romb
 "*" → ★ yulduz
 "+" → + plyus
 "x" → x krest

Linestyle:

"_" → — to'liq chiziq
 "--" → - - kesma
 "-." → -.- nuqta-chiziq
 ":" → ... nuqtali

annotate() vs text():

text() → koordinatlar bo'yicha yozish
 annotate() → nuqtaga nisbatan yozish (offset) ✓
 xytext=(0, 10) → markerdan 10 piksel yuqori

💡 **Eslab qol:** ax.plot() da label= yozilmasa — legend() chaqirilsa ham **bo'sh** chiqadi. Har plot() ga label= berish shart. Shuningdek plt.show() **birinchi** chaqirilgunga qadar barcha sozlamalar qo'llaniladi — show() dan keyin yozilgan set_title() kabi buyruqlar **ishlamaydi**.

● SAVOL 54:

[SCIKIT-LEARN] IRIS DATASET USTIDA LOGISTIC REGRESSION MODELI O'RGATING.
ACCURACY VA CONFUSION MATRIX NI HISOBLANG VA NATIJANI MATPLOTLIB BILAN
VIZUALIZATSIYA QILING.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from sklearn.datasets      import load_iris
from sklearn.linear_model  import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import (
    accuracy_score,
    confusion_matrix,
    classification_report,
)

# =====
# 1. DATASET YUKLASH
# =====

# load_iris() → Iris datasetini yuklaydi
# X → 150 ta o'simlik, 4 ta xususiyat (cm):
#     [sepal length, sepal width, petal length, petal width]
# y → 3 ta sinf:
#     0=Setosa, 1=Versicolor, 2=Virginica
iris = load_iris()
X     = iris.data           # (150, 4) – xususiyatlar
y     = iris.target         # (150,) – sinflar
nomlar = iris.target_names # ["setosa", "versicolor", "virginica"]
xususiyatlar = iris.feature_names

print("=" * 52)
print("  IRIS DATASET:")
print("=" * 52)
print(f"  Jami namuna      : {X.shape[0]} ta")
print(f"  Xususiyatlar soni: {X.shape[1]} ta")
print(f"  Sinflar          : {list(nomlar)}")
print(f"  Har sinfdan      : {np.bincount(y)} ta")

# =====
# 2. TRAIN / TEST BO'LISH
# =====

# test_size=0.2 → 20% test, 80% train
# random_state → takrorlanuvchi natija uchun urug' (seed)
# stratify=y → har sinfdan teng nisbatda bo'lish kafolati
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.2,
    random_state = 42,
```

```

        stratify      = y,
    )

print("\n" + "=" * 52)
print("  TRAIN/TEST BO'LISH:")
print("=" * 52)
print(f"  Train namunalar: {len(X_train)} ta (80%)")
print(f"  Test namunalar : {len(X_test)} ta (20%)")

# =====
# 3. MASSHTABLASH (STANDARTLASHTIRISH)
# =====

# StandardScaler → har xususiyatni o'rtacha=0, std=1 ga keltiradi
# Logistic Regression gradientga asoslangan → masshtab muhim!
# fit()          → faqat train dan o'rtacha va std hisoblash
# transform()    → hisoblangan qiymatlar asosida o'zgartirish
# MUHIM: test uchun FAQAT transform() – fit() emas!
# Aks holda "data leakage" bo'ladi
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # O'rganish + o'zgartirish
X_test  = scaler.transform(X_test)     # Faqat o'zgartirish

# =====
# 4. MODEL YARATISH VA O'RGATISH
# =====

# max_iter → konvergensiya uchun maksimum iteratsiya
# C        → regularizatsiya kuchi (kichik = kuchli regularizatsiya)
#          → Overfitting oldini oladi
# solver   → optimizatsiya algoritmi
#          → "lbfgs" → kichik datasetlar uchun yaxshi
model = LogisticRegression(
    max_iter      = 200,
    C              = 1.0,
    solver        = "lbfgs",
    random_state  = 42,
)

# fit() → modelni train ma'lumotlari asosida o'rgatish
model.fit(X_train, y_train)
print("\n" + "=" * 52)
print("  MODEL O'RGATILDI ✓")
print("=" * 52)

# =====
# 5. BASHORAT VA ACCURACY
# =====

# predict() → har namuna uchun sinf bashorati (0, 1 yoki 2)
y_pred = model.predict(X_test)

# accuracy_score → to'g'ri bashoratlar ulushi
# 1.0 = 100% to'g'ri
accuracy = accuracy_score(y_test, y_pred)

print(f"\n  Accuracy: {accuracy:.4f}  ({accuracy*100:.1f}%)")

```

```

# classification_report → precision, recall, f1-score
print("\n" + "=" * 52)
print("  CLASSIFICATION REPORT:")
print("=" * 52)
print(classification_report(
    y_test, y_pred,
    target_names = nomlar,
))

# =====
# 6. CONFUSION MATRIX
# =====

# confusion_matrix → qaysi sinf qaysi sinfga noto'g'ri bashorat
# Diagonal → to'g'ri bashoratlar
# Diagonal tashqari → xatolar
cm = confusion_matrix(y_test, y_pred)

print("=" * 52)
print("  CONFUSION MATRIX:")
print("=" * 52)
print(f"  {'':12}", end="")
for n in nomlar:
    print(f"  {n[:10]:10}", end="")
print()
for i, qator in enumerate(cm):
    print(f"  {nomlar[i]:12}", end="")
    for val in qator:
        print(f"  {val:<10}", end="")
    print()

# =====
# 7. VIZUALIZATSIYA
# =====

fig = plt.figure(figsize=(14, 5), facecolor="#0f1117")

# GridSpec → ikki grafik: Confusion Matrix + Bar chart
gs = gridspec.GridSpec(1, 2, figure=fig, wspace=0.35)

# — 7a. Confusion Matrix (Heatmap) —————

ax1 = fig.add_subplot(gs[0])
ax1.set_facecolor("#1a1d27")

# imshow() → matritsa qiymatlarini rang sifatida ko'rsatish
# cmap      → rang sxemasi
im = ax1.imshow(cm, cmap="Blues", aspect="auto")

# Colorbar → rang shkalasi
cbar = fig.colorbar(im, ax=ax1, fraction=0.046, pad=0.04)
cbar.ax.yaxis.set_tick_params(color="#8892b0")
plt.setp(cbar.ax.yaxis.get_ticklabels(), color="#8892b0")

# O'q nomlari va belgilari
ax1.set_xticks(range(len(nomlar)))

```

```

ax1.set_yticks(range(len(nomlar)))
ax1.set_xticklabels(
    [n.capitalize() for n in nomlar],
    fontsize=10, color="#cdd6f4", rotation=15,
)
ax1.set_yticklabels(
    [n.capitalize() for n in nomlar],
    fontsize=10, color="#cdd6f4",
)

# Har katak ichiga qiymat yozish
umumiy = cm.sum()
for i in range(len(nomlar)):
    for j in range(len(nomlar)):
        qiymat = cm[i, j]
        # To'q fon bo'lsa oq yoz, och fon bo'lsa qora yoz
        matn_rangi = "white" if qiymat > cm.max() / 2 else "#cdd6f4"
        foiz = qiymat / umumiy * 100
        ax1.text(
            j, i,
            f"{qiymat}\n({foiz:.1f}%)",
            ha = "center",
            va = "center",
            color = matn_rangi,
            fontsize = 11,
            fontweight= "bold",
        )

ax1.set_xlabel(
    "Bashorat qilingan sinf",
    color="#8892b0", fontsize=11, labelpad=10,
)
ax1.set_ylabel(
    "Haqiqiy sinf",
    color="#8892b0", fontsize=11, labelpad=10,
)
ax1.set_title(
    "Confusion Matrix",
    color="#e2e8f0", fontsize=13,
    fontweight="bold", pad=14,
)
for spine in ax1.spines.values():
    spine.set_edgecolor("#2d3148")

```

— 7b. Har sinf uchun to'g'rilik (Bar chart) —————

```

ax2 = fig.add_subplot(gs[1])
ax2.set_facecolor("#1a1d27")

# Har sinf uchun alohida accuracy hisoblash
sinf_accuracy = []
for i in range(len(nomlar)):
    # Faqat shu sinfga tegishli test namunalari
    mask = y_test == i
    sinf_acc = accuracy_score(y_test[mask], y_pred[mask])
    sinf_accuracy.append(sinf_acc * 100)

```

```

ranglar = ["#7c85f3", "#f38ba8", "#a6e3a1"]
xpos    = np.arange(len(nomlar))

bars = ax2.bar(
    xpos, sinf_accuracy,
    color    = ranglar,
    width    = 0.5,
    zorder   = 3,
    edgecolor = "#0f1117",
    linewidth = 1.2,
)

# Har ustun ustiga foiz yozish
for bar, val in zip(bars, sinf_accuracy):
    ax2.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 1.5,
        f"{val:.1f}%",
        ha      = "center",
        va      = "bottom",
        color    = "#e2e8f0",
        fontsize = 12,
        fontweight="bold",
    )

# Umumiy accuracy – gorizonttal chiziq
ax2.axhline(
    y      = accuracy * 100,
    color   = "#fab387",
    linestyle = "--",
    linewidth = 1.8,
    label    = f"Umumiy: {accuracy*100:.1f}%",
    zorder   = 4,
)

ax2.set_xticks(xpos)
ax2.set_xticklabels(
    [n.capitalize() for n in nomlar],
    fontsize=10, color="#cdd6f4",
)
ax2.set_ylim(0, 115)
ax2.set_ylabel(
    "Accuracy (%)",
    color="#8892b0", fontsize=11, labelpad=10,
)
ax2.set_title(
    f"Sinf bo'yicha Accuracy | Jami: {accuracy*100:.1f}%",
    color="#e2e8f0", fontsize=13,
    fontweight="bold", pad=14,
)
ax2.grid(
    axis="y", color="#2d3148",
    linestyle="--", alpha=0.6, zorder=1,
)
ax2.legend(
    fontsize=10, facecolor="#252836",
    edgecolor="#363a52", labelcolor="#cdd6f4",

```

```

)
ax2.yaxis.set_tick_params(labelcolor="#8892b0")
for spine in ax2.spines.values():
    spine.set_edgecolor("#2d3148")

plt.suptitle(
    "Iris Dataset – Logistic Regression Natijalari",
    fontsize = 15,
    fontweight= "bold",
    color      = "#e2e8f0",
    y          = 1.02,
)

plt.tight_layout()
plt.show()

```

Chiqish Natijasi

```

=====
IRIS DATASET:
=====
Jami namuna      : 150 ta
Xususiyatlar soni: 4 ta
Sinflar          : ['setosa', 'versicolor', 'virginica']
Har sinfdan      : [50 50 50] ta

=====
TRAIN/TEST BO'LISH:
=====
Train namunalar: 120 ta (80%)
Test namunalar : 30 ta (20%)

Accuracy: 1.0000 (100.0%)

=====
CLASSIFICATION REPORT:
=====

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30

```

=====
CONFUSION MATRIX:
=====

```

	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	10	0
virginica	0	0	10

Dastur Tuzilishi

Dastur

Dataset yuklash	
load_iris()	→ X (150×4), y (150,)
Train/Test bo'lish	
train_test_split()	→ 80% / 20%, stratify
Masshtablash	
StandardScaler	→ fit(train) → transform(ikkalasi)
Model	
LogisticRegression	→ fit() → predict()
Baholash	
accuracy_score()	→ umumiy to'g'rilik
classification_report()	→ precision/recall/f1
confusion_matrix()	→ qayerda xato
Vizualizatsiya	
imshow(cm)	→ Confusion Matrix heatmap
bar(sinf_accuracy)	→ Sinf bo'yicha accuracy

Asosiy Texnik Tushunchalar

Train/Test bo'lish:

stratify=y → Har sinfdan teng ulush saqlanadi
random_state=42 → Har safar bir xil bo'linish

Strategiysiz:

Train: 60 Setosa
 30 Versicolor
 30 Virginica

Stratify bilan:

Train: 40 Setosa
 40 Versicolor
 40 Virginica ✓

StandardScaler – nima uchun?

Xom ma'lumot: Masshtablangan:
sepal_length: 4.3–7.9 → -2.0 – +2.0
petal_width: 0.1–2.5 → -2.0 – +2.0
Gradient descent teng qadamda yuradi → tezroq

Data Leakage – nima?

XATO: scaler.fit_transform(X_test)
 → Test statistikasi modelga "sizib o'tadi"
 → Haqiqiy accuracy dan yuqori ko'rinadi (aldamchi!)
TO'G'RI: scaler.fit(X_train) → faqat train dan o'rganish
 scaler.transform(X_test) → o'sha qiymatlar bilan o'zgartirish

Confusion Matrix o'qish:

Bashorat→	Set	Ver	Vir	
Haqiqiy ↓				
Setosa	[10	0	0]	← 10 ta to'g'ri
Versicolor	[0	10	0]	← 10 ta to'g'ri
Virginica	[0	0	10]	← 10 ta to'g'ri

Diagonal = to'g'ri bashorat

Tashqarisi = xato (0 bo'lsa – mukammal model)

Metrikalar:

Precision = $TP / (TP + FP)$ → bashorat qilganlarning qanchas to'g'ri

Recall = $TP / (TP + FN)$ → haqiqiylarning qanchasi topildi

F1-score = $2 * P * R / (P + R)$ → ikkalasining balans

Accuracy = to'g'ri / jami → umumiy to'g'rilik

💡 **Eslab qol:** Iris dataset juda **toza** va **kichik** — shuning uchun 100% accuracy ajablanarli emas. Haqiqiy loyihalarda **accuracy yolg'iz metrika emas**: sinflar noto'g'ri taqsimlangan bo'lsa (masalan 95% bir sinf), model hamma narsani bitta sinfga bashorat qilib 95% accuracy olishi mumkin — lekin bu **ishlamaydigan model**. Bunday hollarda **F1-score** yoki **ROC-AUC** ishlatiladi.

● SAVOL 55:

[SCIKIT-LEARN] UY NARXLARINI BASHORAT QILUVCHI ODDIY LINEAR REGRESSION MODELINI TUZING. MSE VA R^2 NI HISOBLANG, HAQIQIY VA BASHORAT QILINGAN QIYMATLARNI GRAFIKDA KO'RSATING.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from sklearn.datasets      import fetch_california_housing
from sklearn.linear_model  import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics       import mean_squared_error, r2_score

# =====
# 1. DATASET YUKLASH
# =====

# fetch_california_housing() → California uy narxlari dataseti
# 20 640 ta namuna, 8 ta xususiyat
# Narx: median uy narxi (100 000 dollar birligida)
#
# Xususiyatlar:
#   MedInc      → o'rtacha daromad
#   HouseAge    → uyning yoshi
#   AveRooms    → o'rtacha xonalar soni
#   AveBedrms   → o'rtacha yotoqxonalar soni
#   Population  → aholi soni
#   AveOccup    → o'rtacha band xonalar
#   Latitude    → kenglik
#   Longitude   → uzunlik

malumot      = fetch_california_housing()
X            = malumot.data                # (20640, 8)
y            = malumot.target              # (20640,) — narx
xususiyatlar = malumot.feature_names

print("=" * 55)
print(" CALIFORNIA HOUSING DATASET:")
print("=" * 55)
print(f" Jami namuna      : {X.shape[0]:,} ta")
print(f" Xususiyatlar soni: {X.shape[1]} ta")
print(f" Narx oralig'i     : ${y.min():.2f} - ${y.max():.2f} (×100k)")
print(f" O'rtacha narx     : ${y.mean():.2f} (×100k)")
```

```

# =====
# 2. TRAIN / TEST BO'LISH
# =====

# test_size=0.2 → 20% test (4128 ta), 80% train (16512 ta)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.2,
    random_state = 42,
)

print("\n" + "=" * 55)
print("  TRAIN/TEST BO'LISH:")
print("=" * 55)
print(f"  Train: {len(X_train):,} ta namuna")
print(f"  Test : {len(X_test):,} ta namuna")

# =====
# 3. MASSHTABLASH
# =====

# LinearRegression gradientga sezgir → masshtab muhim
# fit_transform → train dan o'rganib, o'zgartiradi
# transform      → test ni FAQAT o'zgartiradi (data leakage yo'q)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# =====
# 4. MODEL O'RGATISH
# =====

# LinearRegression →  $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_8x_8$ 
# fit() → eng kichik xatoni beruvchi w koeffitsientlarni topadi
# Yechim: Normal Equation →  $w = (X^T X)^{-1} X^T y$ 
model = LinearRegression()
model.fit(X_train, y_train)

print("\n" + "=" * 55)
print("  MODEL KOEFFITSIENTLARI:")
print("=" * 55)
for nom, koef in zip(xususiyatlar, model.coef_):
    # Musbat koef → narxni oshiradi, manfiy → kamaytiradi
    belgi = "↑" if koef > 0 else "↓"
    print(f"  {nom:<12}: {koef:+.4f}  {belgi}")
print(f"  {'Intercept':<12}: {model.intercept_:+.4f}")

# =====
# 5. BASHORAT VA METRIKALAR
# =====

y_pred = model.predict(X_test)

# MSE → Mean Squared Error (O'rtacha kvadratik xato)
# Haqiqiy va bashorat orasidagi farqlarning kvadrat o'rtachasi
# Birlik: dollar2 → past bo'lgan sari yaxshi

```

```

mse = mean_squared_error(y_test, y_pred)

# RMSE → MSE ning kvadrat ildizi → narx bilan bir birlikda
# Talqin: o'rtacha shu qadar dollar xato qilinadi
rmse = np.sqrt(mse)

# R² → Determinatsiya koeffitsienti (0 dan 1 gacha)
# 1.0 → model barcha dispersiyani tushuntiradi (mukammal)
# 0.0 → model o'rtachadan yaxshiroq emas
# < 0 → model o'rtachadan ham yomon
r2 = r2_score(y_test, y_pred)

# Residuallar → haqiqiy - bashorat (xato qiymatlari)
residuallar = y_test - y_pred

print("\n" + "=" * 55)
print("  MODEL NATIJALARI:")
print("=" * 55)
print(f"  MSE   : {mse:.4f}")
print(f"  RMSE  : {rmse:.4f}  (≈ ${rmse*100:.0f} xato)")
print(f"  R²    : {r2:.4f}  ({r2*100:.1f}% dispersiya tushuntirildi)")

# =====
# 6. VIZUALIZATSIYA – 3 grafik
# =====

fig = plt.figure(figsize=(16, 5), facecolor="#0f1117")
gs = gridspec.GridSpec(1, 3, figure=fig, wspace=0.38)

RANG_NUQTA = "#7c85f3"
RANG_CHIZIQ = "#f38ba8"
RANG_YASHIL = "#a6e3a1"
RANG_SARIQ = "#fab387"

# — 6a. Haqiqiy vs Bashorat qilingan —————

ax1 = fig.add_subplot(gs[0])
ax1.set_facecolor("#1a1d27")

# scatter → har test namunasi uchun (haqiqiy, bashorat) nuqta
# alpha → shaffoflik (ko'p nuqta ustma-ust tushganda ko'rinsin)
ax1.scatter(
    y_test, y_pred,
    color = RANG_NUQTA,
    alpha = 0.35,
    s = 18,          # Nuqta o'lchami
    zorder = 3,
    label = "Namunalar",
)

# Ideal bashorat chizig'i: y = x (haqiqiy = bashorat bo'lsa)
# Nuqtalar bu chiziqqa yaqin bo'lsa → model yaxshi
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
ax1.plot(
    [min_val, max_val],
    [min_val, max_val],

```

```

        color      = RANG_CHIZIQ,
        linewidth  = 2,
        linestyle  = "--",
        zorder     = 4,
        label      = "Ideal chiziq (y=x)",
    )

    ax1.set_xlabel(
        "Haqiqiy narx (x$100k)",
        color="#8892b0", fontsize=10, labelpad=8,
    )
    ax1.set_ylabel(
        "Bashorat narxi (x$100k)",
        color="#8892b0", fontsize=10, labelpad=8,
    )
    ax1.set_title(
        f"Haqiqiy vs Bashorat\nR2 = {r2:.4f}",
        color="#e2e8f0", fontsize=12,
        fontweight="bold", pad=12,
    )
    ax1.legend(
        fontsize=9, facecolor="#252836",
        edgecolor="#363a52", labelcolor="#cdd6f4",
    )
    ax1.grid(
        color="#2d3148", linestyle="--",
        alpha=0.5, zorder=1,
    )
    ax1.tick_params(colors="#8892b0")
    for spine in ax1.spines.values():
        spine.set_edgecolor("#2d3148")

```

— 6b. Residuallar tarqalishi —————

```

ax2 = fig.add_subplot(gs[1])
ax2.set_facecolor("#1a1d27")

```

Residual plot → xatolarning bashorat bo'yicha tarqalishi
Yaxshi model: nuqtalar 0 atrofida tasodifiy tarqalgan bo'lishi kerak
Pattern ko'rinsa → model biror narsani o'tkazib yuborgan

```

ax2.scatter(
    y_pred, residuallar,
    color = RANG_YASHIL,
    alpha = 0.35,
    s     = 18,
    zorder = 3,
    label = "Residuallar",
)

```

Nol chiziq → ideal residual = 0

```

ax2.axhline(
    y      = 0,
    color  = RANG_CHIZIQ,
    linewidth = 2,
    linestyle = "--",
    zorder  = 4,
    label   = "Residual = 0",
)

```

```

)

# ±1 RMSE zonasi – ko'p namuna shu ichida bo'lishi kerak
ax2.axhspan(
    -rmse, rmse,
    alpha    = 0.08,
    color     = RANG_SARIQ,
    zorder    = 2,
    label     = f"±RMSE zona ({rmse:.2f})",
)

ax2.set_xlabel(
    "Bashorat narxi (x$100k)",
    color="#8892b0", fontsize=10, labelpad=8,
)
ax2.set_ylabel(
    "Residual (Haqiqiy - Bashorat)",
    color="#8892b0", fontsize=10, labelpad=8,
)
ax2.set_title(
    f"Residuallar Tarqalishi\nRMSE = {rmse:.4f}",
    color="#e2e8f0", fontsize=12,
    fontweight="bold", pad=12,
)
ax2.legend(
    fontsize=9, facecolor="#252836",
    edgecolor="#363a52", labelcolor="#cdd6f4",
)
ax2.grid(
    color="#2d3148", linestyle="--",
    alpha=0.5, zorder=1,
)
ax2.tick_params(colors="#8892b0")
for spine in ax2.spines.values():
    spine.set_edgecolor("#2d3148")

```

— 6c. Xususiyat koeffitsientlari —————

```

ax3 = fig.add_subplot(gs[2])
ax3.set_facecolor("#1a1d27")

# Koeffitsientlarni kattadan kichikka saralash
tartib = np.argsort(model.coef_)
nom_lar = [xususiyatlar[i] for i in tartib]
koef_lar = model.coef_[tartib]

# Musbat → yashil, Manfiy → qizil
bar_ranglar = [
    RANG_YASHIL if k > 0 else RANG_CHIZIQ
    for k in koef_lar
]

bars = ax3.barh(
    nom_lar, koef_lar,
    color     = bar_ranglar,
    edgecolor = "#0f1117",
    linewidth = 0.8,
)

```

```

        zorder      = 3,
    )

    # Har ustun yoniga qiymat yozish
    for bar, val in zip(bars, koef_lar):
        xpos = val + (0.02 if val >= 0 else -0.02)
        ha    = "left" if val >= 0 else "right"
        ax3.text(
            xpos,
            bar.get_y() + bar.get_height() / 2,
            f"{val:+.3f}",
            va      = "center",
            ha      = ha,
            color    = "#cdd6f4",
            fontsize = 9,
        )

    ax3.axvline(
        x          = 0,
        color       = "#45475a",
        linewidth   = 1.2,
        linestyle   = "-",
        zorder      = 2,
    )

    ax3.set_xlabel(
        "Koeffitsient qiymati",
        color="#8892b0", fontsize=10, labelpad=8,
    )

    ax3.set_title(
        "Xususiyat Koeffitsientlari\n(Masshtablangan)",
        color="#e2e8f0", fontsize=12,
        fontweight="bold", pad=12,
    )

    ax3.grid(
        axis="x", color="#2d3148",
        linestyle="--", alpha=0.5, zorder=1,
    )

    ax3.tick_params(colors="#8892b0")
    for spine in ax3.spines.values():
        spine.set_edgecolor("#2d3148")

    plt.suptitle(
        "California Housing – Linear Regression Natijalari",
        fontsize    = 14,
        fontweight   = "bold",
        color        = "#e2e8f0",
        y            = 1.02,
    )

    plt.tight_layout()
    plt.show()

```

Chiqish Natijasi

```

=====
CALIFORNIA HOUSING DATASET:
=====
Jami namuna      : 20,640 ta
Xususiyatlar soni: 8 ta
Narx oralig'i    : $0.15 - $5.00 (x100k)
O'rtacha narx    : $2.07 (x100k)
=====

MODEL KOEFFITSIENTLARI:
=====
MedInc      : +0.8296  ↑
HouseAge    : +0.1193  ↑
AveRooms    : +0.2769  ↑
AveBedrms   : -0.2975  ↓
Population  : -0.0043  ↓
AveOccup    : -0.0393  ↓
Latitude    : -0.8993  ↓
Longitude   : -0.8705  ↓
Intercept   : 2.0686
=====

MODEL NATIJALARI:
=====
MSE   : 0.5559
RMSE  : 0.7456  (≈ $74,560 xato)
R2 : 0.5758  (57.6% dispersiya tushuntirildi)

```

Dastur Tuzilishi

Dastur

- Dataset yuklash
 - fetch_california_housing() → X (20640×8), y
- Train/Test bo'lish
 - train_test_split() → 80% / 20%
- Masshtablash
 - StandardScaler → fit(train) → transform
- Model
 - LinearRegression → fit() → predict()
- Metrikalar
 - mean_squared_error() → MSE
 - np.sqrt(MSE) → RMSE
 - r2_score() → R²
- Vizualizatsiya
 - scatter(y_test, y_pred) → Haqiqiy vs Bashorat
 - scatter(y_pred, residual) → Residual plot
 - barh(koeffitsientlar) → Feature importance

Asosiy Texnik Tushunchalar

Linear Regression formulasi:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_8x_8$$

xususiyatlar

	intercept (bo'sh a'zo)
	bashorat

Maqsad: $\Sigma(y_i - \hat{y}_i)^2 \rightarrow \text{MINIMUM}$

Yechim: Normal Equation $\rightarrow w = (X^T X)^{-1} X^T y$

MSE vs RMSE vs R^2 :

MSE = $\Sigma(\text{haqiqiy} - \text{bashorat})^2 / n$
 $\rightarrow$ katta xatolarni ko'proq jazolanadi (kvadrat!)
 $\rightarrow$ birlik: dollar² (talqin qiyin)

RMSE = $\sqrt{\text{MSE}}$
 $\rightarrow$ dollar birligida $\rightarrow$ "o'rtacha \$74k xato"
 $\rightarrow$ MSE dan tushunarli

$R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$
 $\text{SS}_{\text{res}} = \Sigma(y - \hat{y})^2 \leftarrow$ model xatosi
 $\text{SS}_{\text{tot}} = \Sigma(y - \bar{y})^2 \leftarrow$ o'rtachadan farq
 0.576 $\rightarrow$ model 57.6% ni tushuntirdi
 42.4% tushuntirilmagan $\rightarrow$ boshqa omillar bor

Residual plot talqini:

YAXSHI model:	YOMON model:
<pre> — </pre>	<pre> — </pre>
Tasodifiy tarqalgan	Pattern ko'rinmoqda
$\rightarrow$ model to'g'ri	$\rightarrow$ chiziqli emas yoki o'tkazib yuborilgan omil

Koeffitsient talqini (masshtablangan):

MedInc: +0.83 $\rightarrow$ daromad oshsa narx oshadi (eng muhim!)
 Latitude: -0.90 $\rightarrow$ shimolga borsa narx tushadi
 AveBedrms: -0.30 $\rightarrow$ ko'p yotoqxona $\rightarrow$ arzonroq (paradoks!)
 chunki AveRooms bilan korrelyatsiya bor

💡 **Eslab qol:** $R^2 = 0.576$ ko'rinishida past tuyulishi mumkin, lekin bu **chiziqli model chegarasi** — uy narxiga ta'sir qiluvchi omillar (manzil obro'si, ta'mir holati, bozor trendi) datasetda yo'q. Yaxshiroq natija uchun **Random Forest** yoki **Gradient Boosting** ishlatiladi — ular nochiziqli bog'liqliklarni ham ushlaydi va odatda $R^2 = 0.80+$ beradi.